



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Lahari Bhargav
Student Name	Dhatrika Deekshitha
Roll Number	2520030216
Branch	CSE
Course Outcome	CO2

Case Study Topic: Parking Revenue Prefix Sums Using Fenwick Tree (BIT)

Work Summary:

In this case study, we built a **Fenwick Tree (also called Binary Indexed Tree or BIT)** for **ParkSmart Intelligent Smart Parking Management System**. The system stores **daily parking revenue collections** in an array `revenue[1..365]` for each parking facility. Two main operations are needed: **(i) point update** — add revenue to today's collection whenever a vehicle completes parking payment, and **(ii) range-sum query** — find the total parking revenue between two dates for financial reports and revenue analysis. Both operations work in **$O(\log n)$** time.

We used the first **15 days of parking revenue data** as input, built the BIT array step by step, answered the query for **total parking revenue from Jan 5 to Jan 12**, found and fixed a bug in a given **`prefixSum()`** code, and explained why a **BIT is better than a Segment Tree** for this workload.

Implementation Details:

1. The revenue array for Jan 1–15 is:
`revenue = [0, 1200, 800, 0, 2400, 1500, 600, 0, 0, 3500, 0, 1100, 950, 700, 0]`
(index 0 is unused).
2. Each BIT cell `bit[i]` stores the sum of a range whose length equals $2^{\text{lsb}(i)}$, where `lsb` means the lowest set bit of `i`.
3. To answer the query **total parking revenue Jan 5–12**, we calculate:
 $\text{prefix}(12) - \text{prefix}(4) = 12050 - 4400 = ₹7,650$.
4. The bug in **FenwickBuggy** is in the `prefixSum()` method:
`i -= (i & -i) + 1` should be changed to
`i -= (i & -i)`.
The extra `+1` causes the index to jump incorrectly.
5. BIT is better than a Segment Tree for this workload because it uses **less memory (n words vs $\sim 4n$)**, accesses memory more efficiently, and requires fewer lines of code.

(a) BIT Construction and Query [5 Marks — CO2, BTL3]

Index: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Revenue : 1200 800 0 2400 1500 600 0 0 3500 0 1100 950 700 0 0

How BIT cells are calculated:

Each $\text{bit}[i]$ covers a range of indices. The range length = $2^{(\text{lowest set bit of } i)}$.

$\text{bit}[4]$ — non-trivial cell:

- $\text{lsb}(4) = 4$, so $\text{bit}[4]$ covers 4 values: revenue [1] to revenue [4]
- $\text{bit}[4] = 1200 + 800 + 0 + 2400 = 4400$

$\text{bit}[8]$ — non-trivial cell:

- $\text{lsb}(8) = 8$, so $\text{bit}[8]$ covers 8 values: revenue [1] to revenue [8]

- $\text{bit}[8] = 1200 + 800 + 0 + 2400 + 1500 + 600 + 0 + 0 = 6500$

$\text{bit}[12]$ — non-trivial cell:

- $\text{lsb}(12) = 4$, so $\text{bit}[12]$ covers 4 values: revenue [9] to revenue [12]
- $\text{bit}[12] = 3500 + 0 + 1100 + 950 = 5550$

Full BIT array:

i:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
bit[i]:	1200	2000	0	4400	1500	2100	0	6500	3500	3500	1100	5550	700	700	0

Computing the Query: $\Sigma \text{ revenue } [5..12] = \text{prefix}(12) - \text{prefix}(4)$

Step 1 — prefix(12):

- $i = 12 \rightarrow \text{add } \text{bit}[12] = 5550$; next $i = 12 - (12 \& -12) = 12 - 4 = 8$
- $i = 8 \rightarrow \text{add } \text{bit}[8] = 6500$; next $i = 8 - 8 = 0 \rightarrow \text{stop}$
- $\text{prefix}(12) = 5550 + 6500 = 12050$
- BIT cells visited: $\text{bit}[12]$, $\text{bit}[8]$

Step 2 — prefix(4):

- $i = 4 \rightarrow \text{add } \text{bit}[4] = 4400$; next $i = 4 - 4 = 0 \rightarrow \text{stop}$
- $\text{prefix}(4) = 4400$
- BIT cells visited: $\text{bit}[4]$

Answer: $12050 - 4400 = ₹7,650$

Verification: Total parking revenue collected from Jan 5–12:
 $1500 + 600 + 0 + 0 + 3500 + 0 + 1100 + 950 = ₹7,650$

(b) Bug Identification and Fix [6 Marks — CO2, BTL4]

The Bug:

In the prefix Sum method, the line is written as:

```
i -= (i & -i) + 1; // wrong
```

The correct line should be:

```
i -= (i & -i); // correct
```

Why this is a bug:

In a Fenwick Tree, to move to the previous BIT cell we must subtract the lowest set bit (lsb) of i from i . The buggy code adds an extra 1 after subtracting the lsb, which makes i jump to the wrong position. This causes some BIT cells to be skipped, giving a wrong prefix sum.

Which inputs trigger the bug:

The bug gives wrong answers whenever the traversal path includes an odd index. For example:

- $\text{prefix}(1)$: $i=1$, $\text{lsb}=1$, buggy code does $i = 1 - 1 - 1 = -1$, loop exits wrongly.
- $\text{prefix}(3)$: $i=3$, $\text{lsb}=1$, buggy code jumps to $i=1$ instead of $i=2$, so $\text{bit}[2]$ is skipped.

The bug gives correct answers only when all cells in the traversal path happen to store zero.

Time Complexity: $O(\log n)$ for both prefix Sum and point Update — the fix does not change the time complexity.

Java Code:

```
class ParkSmartFenwickTree {

    int[] bit;
    int n;

    ParkSmartFenwickTree(int n) {
        this.n = n;
        this.bit = new int[n + 1];
    }

    // Return revenue[1] + revenue[2] + ... + revenue[i]
    long prefixSum(int i) {
        long sum = 0;

        while (i > 0) {
            sum += bit[i];
            i -= (i & -i); // FIXED
        }

        return sum;
    }

    // Add delta to revenue at index i
    void pointUpdate(int i, int delta) {
        while (i <= n) {
            bit[i] += delta;
            i += (i & -i);
        }
    }

    public static void main(String[] args) {

        ParkSmartFenwickTree fen = new ParkSmartFenwickTree(15);

        // Daily Parking Revenue (Jan 1 - Jan 15)
        int[] revenue = {
            0,
            1200, 800, 0, 2400, 1500,
            600, 0, 0, 3500, 0,
```

```

        1100, 950, 700, 0, 0
    };

    // Build BIT
    for (int i = 1; i <= 15; i++) {
        fen.pointUpdate(i, revenue[i]);
    }

    long p12 = fen.prefixSum(12);
    long p4 = fen.prefixSum(4);

    System.out.println("=====");
    System.out.println(" ParkSmart Revenue Analysis");
    System.out.println("=====");

    System.out.println("Prefix Revenue (Jan 1 - Jan 12) = Rs." + p12);
    System.out.println("Prefix Revenue (Jan 1 - Jan 4) = Rs." + p4);

    System.out.println(
        "Parking Revenue Jan 5 - Jan 12 = Rs."
        + (p12 - p4)
    );
}
}

```

(c) BIT vs Segment Tree for this Workload [4 Marks — CO2, BTL5]

Why BIT is better for ParkSmart:

The most important point is that this workload has NO range updates. Every parking payment is a single point update (add revenue to one day only). This is exactly the situation where BIT works perfectly and there is no reason to use a heavier data structure like a Segment Tree.

Three quantitative reasons:

(i) Memory:

- BIT uses n words of memory (one array of size n).
- Segment Tree uses approximately $4n$ words (needs extra nodes for internal tree structure).
- For large smart parking facilities tracking daily revenue records throughout the year, BIT saves roughly 3x memory. This is very important during the 6-hour batch processing window.

(ii) Cache behaviour during the 50M-customer batch:

- BIT stores all values in a single flat array `bit[1..n]`. When traversing, it accesses nearby memory locations (e.g., index $12 \rightarrow 8 \rightarrow 0$), which the CPU cache handles very efficiently.
- Segment Tree stores data in tree nodes spread all over memory. Jumping between nodes causes many cache misses, which slows down processing significantly when generating revenue reports and analytics for multiple parking locations.

(iii) Code surface (lines of working code):

- BIT prefix Sum and point Update together need about 10 lines of code.
- Segment Tree needs separate build, update, and query functions — roughly 40 lines.
- Fewer lines means fewer chances for bugs and easier maintenance in the ParkSmart parking management system.

Conclusion: BIT is the right choice for this workload. Since there are no range updates, the extra power of a Segment Tree is not needed, and BIT wins on memory, speed, and simplicity. For ParkSmart, BIT efficiently supports daily parking revenue updates and revenue-report generation while using minimal memory and providing fast query performance.

Two scenarios where Segment Tree would be the better choice:

1. **If range updates are added** : for example, if ParkSmart needs to apply a parking discount or service charge to all revenue records between two dates at once (`spend[l..r] += fee`). BIT cannot handle this easily; a Segment Tree with lazy propagation handles it in $O(\log n)$.
2. **If range maximum queries are needed** : for example, if management needs to find the single highest revenue day within a date range (`max of spend[l..r]`). BIT cannot support range-max; a Segment Tree supports it naturally.

Result: Screenshots / Output

```
=====
ParkSmart Revenue Analysis
=====
Prefix Revenue (Jan 1 - Jan 12) = Rs.12050
Prefix Revenue (Jan 1 - Jan 4)  = Rs.4400
Parking Revenue Jan 5 - Jan 12 = Rs.7650
PS-D:\3rd sem\dsa\
```

GitHub Link:

<https://github.com/dhatrikadeekshitha/DSA>

Student Signature	Faculty Signature
	

